

✓ Estratégias de treino

Até agora no curso, vimos somente uma estratégia de treino para redes neurais: o treinamento do zero. Entretanto, há outras formas de se explorar redes neurais. Nessa aula, vamos rever a estratégia treinamento do zero além de apresentar duas novas formas:

1. rede neural como um extrator de características, e
2. *fine-tuning*.

Para cada uma dessas estratégias, vamos apresentar sua definição, vantagens e desvantagens. Antes, vamos instalar o Pytorch, importar alguns pacotes e definir funções para carregar os dados.

```
import time, os, sys, numpy as np
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F

from torch import optim
from torchsummary import summary

import time, os, sys, numpy as np

# Test if GPU is available, if not, use cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
n = torch.cuda.device_count()
devices_ids = list(range(n))
print(device)
```

cpu

```
def load_data_cifar10(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Cifar10-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.CIFAR10(root=root, train=True, download=True, transform=transformer)
    mnist_test = torchvision.datasets.CIFAR10(root=root, train=False, download=True, transform=transformer)
    num_workers = 0 if sys.platform.startswith('win32') else 4

    train_iter = torch.utils.data.DataLoader(mnist_train,
                                              batch_size=batch_size, shuffle=True,
                                              num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(mnist_test,
                                            batch_size=batch_size, shuffle=False,
                                            num_workers=num_workers)

    return train_iter, test_iter

# funções básicas
def _get_batch(batch):
    """Return features and labels on ctx."""
    features, labels = batch
    if labels.type() != features.type():
        labels = labels.type(features.type())
    return (torch.nn.DataParallel(features, device_ids=devices_ids),
            torch.nn.DataParallel(labels, device_ids=devices_ids), features.shape[0])

# Função usada para calcular acurácia
def evaluate_accuracy(data_iter, net, loss):
    """Evaluate accuracy of a model on the given data set."""

    acc_sum, n, l = torch.Tensor([0]), 0, 0
    net.eval()
    with torch.no_grad():
        for X, y in data_iter:
            #y = y.astype('float32')
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l += loss(y_hat, y).sum()
            acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
```

```

        n += y.size()[0]

    return acc_sum.item() / n, l.item() / len(data_iter)

# Função usada no treinamento e validação da rede
def train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
                  num_epochs):
    print('training on', device)
    for epoch in range(num_epochs):
        net.train()
        train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0, time.time()
        for X, y in train_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            trainer.zero_grad()
            l = loss(y_hat, y).sum()
            l.backward()
            trainer.step()
            train_l_sum += l.item()
            train_acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]
        test_acc, test_loss = evaluate_accuracy(test_iter, net, loss)
        print('epoch %d, train loss %.4f, train acc %.3f, test loss %.4f, '
              'test acc %.3f, time %.1f sec'
              % (epoch + 1, train_l_sum / len(train_iter), train_acc_sum / n, test_loss,
                 test_acc, time.time() - start))

```

› Treinamento do zero

Como dito anteriormente, essa foi a única estratégia vista até o momento no curso. Nessa estratégia, uma rede neural é proposta, **inicializada com pesos aleatórios** e treinada até convergir. A **vantagem** dessa estratégia é liberdade para definir como quiser a arquitetura da rede e seus hiper-parâmetros. Por outro lado, a **desvantagem** é que essa estratégia requer muitos dados para convergir a rede inicializada aleatoriamente. Logo, se tivermos poucos dados, essa não é a estratégia mais recomendada. Abaixo, uma representação visual dessa estratégia.

Apesar de já termos visto essa estratégia na prática, vamos vê-la aqui novamente para efeitos de comparação com as outras técnicas. Para tal, vamos, primeiro, definimos a arquitetura da [AlexNet](#).

↳ 4 células ocultas

✓ Extrator de características

A terceira e última estratégia, mostrada na figura abaixo, é usar uma rede neural pré-treinada em algum dataset grande para extrair características de um outro dataset. Essa estratégia é preferível quando o dataset que se quer extrair as *features* tem muito poucas amostras, inviabilizando o treinamento ou *fine-tuning* da rede.

Existem duas formas de se explorar essa estratégia. A primeira consiste em substituir e treinar somente a última camada da rede neural. Nessa primeira forma, todas as outras camadas da rede ficam com *learning rate* 0, ou seja, não aprendem nada, e são somente usadas como codificadores/extratores de características. A segunda forma, *features* das imagens do dataset que se quer classificar são extraídas da penúltima camada da rede pré-treinada (geralmente, a camada antes da camada de classificação). Essas *features* são então usadas para se treinar um algoritmo externo (como um SVM ou *random forest*), que então classifica o dataset.

```

# rede baseada na AlexNet
net = torchvision.models.alexnet(pretrained=True)

# Sending model to device
net.to(device)

for param in net.parameters():
    param.requires_grad = False

print(summary(net, (3, 227, 227)))

num_ftrs = net.classifier[6].in_features
net.classifier[6] = nn.Linear(num_ftrs, 10) # Alterando a última layer para retornar 10 classes ao invés de 1000

# Sending model to device
net.to(device)

# Verifique no output a última camada do classifier, podemos ver que sua saída é 10
print(net)

```

```
# Podemos ver que este output mostra que apenas 40970 parâmetros serão treinados. Ou seja, somente a última camada
print(summary(net, (3,227,227)))
```

```
# Código retirado de https://pytorch.org/tutorials/beginner/finetuning_torchvision_models_tutorial.html
```

```
# Gather the parameters to be optimized/updated in this run. If we are
# finetuning we will be updating all parameters. However, if we are
# doing feature extract method, we will only update the parameters
# that we have just initialized, i.e. the parameters with requires_grad
# is True.
print("Params to learn: ")
```

```
params_to_update = []
for name,param in net.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param)
    print("\t",name)
```

```
-----
Layer (type)          Output Shape          Param #
-----
Conv2d-1              [-1, 64, 56, 56]      23,296
ReLU-2                [-1, 64, 56, 56]      0
MaxPool2d-3           [-1, 64, 27, 27]      0
Conv2d-4              [-1, 192, 27, 27]     307,392
ReLU-5                [-1, 192, 27, 27]     0
MaxPool2d-6           [-1, 192, 13, 13]     0
Conv2d-7              [-1, 384, 13, 13]     663,936
ReLU-8                [-1, 384, 13, 13]     0
Conv2d-9              [-1, 256, 13, 13]     884,992
ReLU-10               [-1, 256, 13, 13]     0
Conv2d-11             [-1, 256, 13, 13]     590,080
ReLU-12               [-1, 256, 13, 13]     0
MaxPool2d-13          [-1, 256, 6, 6]       0
AdaptiveAvgPool2d-14  [-1, 256, 6, 6]       0
Dropout-15            [-1, 9216]            0
Linear-16              [-1, 4096]            37,752,832
ReLU-17               [-1, 4096]            0
Dropout-18            [-1, 4096]            0
Linear-19              [-1, 4096]            16,781,312
ReLU-20               [-1, 4096]            0
Linear-21              [-1, 1000]            4,097,000
=====
Total params: 61,100,840
Trainable params: 0
Non-trainable params: 61,100,840
-----
Input size (MB): 0.59
Forward/backward pass size (MB): 8.49
Params size (MB): 233.08
Estimated Total Size (MB): 242.16
-----
None
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
```

```
# Treinando a última camada da rede acima
# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 20, 0.001, 100, 0.0001

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
```

```

train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# trainer do gluon
trainer = optim.SGD(params_to_update, lr=lr, weight_decay=wd_lambda, momentum=0.9)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)

```

```

Files already downloaded and verified
Files already downloaded and verified
training on cuda
epoch 1, train loss 1.3262, train acc 0.534, test loss 1.0916, test acc 0.610, time 86.6 sec
epoch 2, train loss 1.1642, train acc 0.588, test loss 0.9961, test acc 0.654, time 86.8 sec
epoch 3, train loss 1.1277, train acc 0.602, test loss 0.9634, test acc 0.662, time 86.9 sec
epoch 4, train loss 1.1151, train acc 0.606, test loss 0.9462, test acc 0.672, time 86.8 sec
epoch 5, train loss 1.0945, train acc 0.614, test loss 0.9265, test acc 0.676, time 86.7 sec
epoch 6, train loss 1.0882, train acc 0.618, test loss 0.9345, test acc 0.671, time 86.8 sec
epoch 7, train loss 1.0796, train acc 0.618, test loss 0.9326, test acc 0.678, time 86.5 sec
epoch 8, train loss 1.0778, train acc 0.619, test loss 0.8997, test acc 0.684, time 86.5 sec
epoch 9, train loss 1.0652, train acc 0.626, test loss 0.9065, test acc 0.679, time 86.6 sec
epoch 10, train loss 1.0652, train acc 0.624, test loss 0.8911, test acc 0.687, time 86.8 sec
epoch 11, train loss 1.0613, train acc 0.623, test loss 0.9190, test acc 0.674, time 87.0 sec
epoch 12, train loss 1.0560, train acc 0.625, test loss 0.9070, test acc 0.678, time 86.6 sec
epoch 13, train loss 1.0552, train acc 0.627, test loss 0.8888, test acc 0.686, time 86.7 sec
epoch 14, train loss 1.0525, train acc 0.627, test loss 0.9140, test acc 0.673, time 86.8 sec
epoch 15, train loss 1.0493, train acc 0.628, test loss 0.8974, test acc 0.684, time 86.7 sec
epoch 16, train loss 1.0452, train acc 0.632, test loss 0.9008, test acc 0.683, time 86.6 sec
epoch 17, train loss 1.0462, train acc 0.631, test loss 0.8753, test acc 0.693, time 86.6 sec
epoch 18, train loss 1.0478, train acc 0.630, test loss 0.8919, test acc 0.681, time 86.8 sec
epoch 19, train loss 1.0452, train acc 0.630, test loss 0.9101, test acc 0.679, time 86.5 sec
epoch 20, train loss 1.0377, train acc 0.634, test loss 0.8619, test acc 0.698, time 86.9 sec

```

```

# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 20, 0.01, 100, 0.0001

# rede baseada na AlexNet-5
net = torchvision.models.alexnet(pretrained=True)

# Sending model to device
net.to(device)

# carregamento do dado: fashion mnist
train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# remove last fully-connected layer
new_classifier = nn.Sequential(*list(net.classifier.children())[:-1])
net.classifier = new_classifier

print(summary(net,(3,227,227))) # visualize number of parameters' net, output of each layer and total mega bytes necessary
                                # and stored weights.

first = True
with torch.no_grad():
    for X, y in train_iter:
        X, y = X.to(device), y.to(device)
        features = net(X)
        if first is True:
            train_features = features.cpu().numpy()
            train_labels = y.cpu().numpy()
            first = False
        else:
            train_features = np.concatenate((train_features, features.cpu().numpy()))
            train_labels = np.concatenate((train_labels, y.cpu().numpy()))

first = True
with torch.no_grad():
    for X, y in test_iter:
        X, y = X.to(device), y.to(device)
        features = net(X)
        if first is True:
            test_features = features.cpu().numpy()
            test_labels = y.cpu().numpy()
            first = False
        else:
            test_features = np.concatenate((test_features, features.cpu().numpy()))
            test_labels = np.concatenate((test_labels, y.cpu().numpy()))

print(train_features.shape, test_features.shape)

```

Files already downloaded and verified
Files already downloaded and verified

Layer (type)	Output Shape	Param #

Conv2d-1	[-1, 64, 56, 56]	23,296
ReLU-2	[-1, 64, 56, 56]	0
MaxPool2d-3	[-1, 64, 27, 27]	0
Conv2d-4	[-1, 192, 27, 27]	307,392
ReLU-5	[-1, 192, 27, 27]	0
MaxPool2d-6	[-1, 192, 13, 13]	0
Conv2d-7	[-1, 384, 13, 13]	663,936
ReLU-8	[-1, 384, 13, 13]	0
Conv2d-9	[-1, 256, 13, 13]	884,992
ReLU-10	[-1, 256, 13, 13]	0
Conv2d-11	[-1, 256, 13, 13]	590,080
ReLU-12	[-1, 256, 13, 13]	0
MaxPool2d-13	[-1, 256, 6, 6]	0
AdaptiveAvgPool2d-14	[-1, 256, 6, 6]	0
Dropout-15	[-1, 9216]	0
Linear-16	[-1, 4096]	37,752,832
ReLU-17	[-1, 4096]	0
Dropout-18	[-1, 4096]	0
Linear-19	[-1, 4096]	16,781,312
ReLU-20	[-1, 4096]	0
=====		

Total params: 57,003,840

Trainable params: 57,003,840

Non-trainable params: 0

Input size (MB): 0.59

Forward/backward pass size (MB): 8.48

Params size (MB): 217.45

Estimated Total Size (MB): 226.52

None

(50000, 4096) (10000, 4096)

```
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score
```

```
clf = LinearSVC()
clf.fit(train_features, train_labels)
```

```
pred = clf.predict(test_features)
print(accuracy_score(test_labels, pred))
```

```
/usr/local/lib/python3.6/dist-packages/sklearn/svm/_base.py:947: ConvergenceWarning: Liblinear failed to converge, increase
"the number of iterations.", ConvergenceWarning)
0.5193
```

› Fine-tuning

A segunda estratégia é chamada de *fine-tuning*, e é comumente classificada como um estratégia de *transfer learning*, onde o aprendizado é transferido entre datasets. Especificamente, esta estratégia, representada na figura abaixo, tenta usar um modelo pré-treinado aprendido anteriormente em algum dataset (geralmente muito grande, como o [ImageNet](#)) para classificar outro conjunto de dados diferentes (geralmente com poucas amostras).

Como esses dados podem possuir características diferentes, treinamos a rede usando um *learning rate* pequeno, apenas para fazer pequenos ajustes nos pesos. Entretanto, como esses datasets geralmente tem número e classes diferentes, a última camada não é usada nessa transferência de peso e, geralmente, é inicializada aleatoriamente (e por isso, tem um *learning rate* mais alto que as demais camadas).

Por fim, é um [fato conhecido](#) que as redes neurais conseguem aprender características de baixo nível nas camadas iniciais. Geralmente, essas características são comuns à vários datasets. Por isso, uma opção durante o processo de *fine-tuning* é "congelar" as camadas iniciais (ou seja, não treiná-las) e treinar somente as demais camadas com taxa de aprendizado bem pequeno (exceto pela camada de classificação).

No bloco de código abaixo, importamos a rede pré-treinada [AlexNet](#), que foi treinada no dataset do [ImageNet](#), que tem 1000 classes. Como iremos fazer *fine-tuning* nessa arquitetura para o dataset do [CIFAR-10](#), que tem somente 10 classes, removeremos a última camada e criaremos uma nova camada inicializada aleatoriamente.

↳ 1 célula oculta

✓ Prática

1. É possível melhorar o resultado obtido anteriormente? Estude o [model_zoo](#) e tente usar as estratégias anteriores com diferentes redes neurais para melhorar o resultado. Algumas redes possíveis:
 - [MobileNets](#)
 - [VGGs](#)
 - [ResNets](#)
 - [DenseNets](#)
2. Procure agora congelar algumas camadas para realizar o *fine-tuning*. Essa estratégia é melhor quando se tem poucas imagens para fazer o *fine-tuning*.
3. Procura usar outros algoritmos de aprendizado de máquina (como [random forest](#) e [SVM-RBF](#)) para classificar *deep features* extraídas de uma rede neural pré-treinada.
4. Procure também extrair e classificar *features* de outras camadas convolucionais.
5. Procure usar as diferentes estratégias para melhorar os resultados dos datasets que já usamos, como [MNIST](#) e [Fashion MNIST](#).

✓ Fine-tuning ResNet-18

```
# rede baseada na AlexNet
net = torchvision.models.resnet18(pretrained=True)

# Sending model to device
net.to(device)

print(net)

num_fts = net.fc.in_features
net.fc = nn.Linear(num_fts, 10) # Alterando a última layer para retornar 10 classes ao invés de 1000

# Sending model to device
net.to(device)

# Verifique no output a última camada do classifier, podemos ver que sua saída é 10
print(net)

# Treinando a última camada da rede acima
# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 20, 0.001, 100, 0.0001

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: cifar 10
train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# trainer
trainer = optim.SGD([
    {'params': net.conv1.parameters(), 'lr': lr * 0.1},
    {'params': net.bn1.parameters(), 'lr': lr * 0.1},
    {'params': net.relu.parameters(), 'lr': lr * 0.1},
    {'params': net.maxpool.parameters(), 'lr': lr * 0.1},
    {'params': net.layer1.parameters(), 'lr': lr * 0.1},
    {'params': net.layer2.parameters(), 'lr': lr * 0.1},
    {'params': net.layer3.parameters(), 'lr': lr * 0.1},
    {'params': net.layer4.parameters(), 'lr': lr * 0.1},
    {'params': net.avgpool.parameters(), 'lr': lr * 0.1},
    {'params': net.fc.parameters(), 'lr': lr}], weight_decay=wd_lambda, momentum=0.9)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
```

```

        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
)
(layer2): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (1): BasicBlock(
    (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
(layer3): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (1): BasicBlock(
    (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)

```

✎ Extracting features ResNet-152

```

# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 20, 0.01, 100, 0.0001

# rede baseada na resnet152
net = torchvision.models.resnet152(pretrained=True)

# Sending model to device
net.to(device)

# carregamento do dado: fashion mnist
train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# remove last fully-connected layer
modules=list(net.children())[:-1]
net=nn.Sequential(*modules)

print(summary(net,(3,227,227))) # visualize number of parameters' net, output of each layer and total mega bytes necessary
                                # and stored weights.

first = True
with torch.no_grad():
    for X, y in train_iter:
        X, y = X.to(device), y.to(device)
        features = net(X)
        if first is True:
            train_features = features.cpu().numpy()
            train_labels = y.cpu().numpy()
            first = False
        else:
            train_features = np.concatenate((train_features, features.cpu().numpy()))
            train_labels = np.concatenate((train_labels, y.cpu().numpy()))

first = True
with torch.no_grad():
    for X, y in test_iter:
        X, y = X.to(device), y.to(device)
        features = net(X)

```

```
if first is True:
    test_features = features.cpu().numpy()
    test_labels = y.cpu().numpy()
    first = False
else:
    test_features = np.concatenate((test_features, features.cpu().numpy()))
    test_labels = np.concatenate((test_labels, y.cpu().numpy()))

print(train_features.shape, test_features.shape)
```

```
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

clf = LinearSVC()
clf.fit(train_features, train_labels)

pred = clf.predict(test_features)
print(accuracy_score(test_labels, pred))
```

